

Neuromorphic Computing on FPGA URSC,ISRO

Daily Study Report — Day 4 to 6

Submitted by
Shaurya Singh

B.Tech Electronics & Communication Engineering
Indian Institute of Space Science and Technology
Thiruvananthapuram | Batch 2024-2028

Topic: N-MNIST Benchmark: SNN vs. CNN —
Implementation & Analysis

Report Date: June 2026

Tools: Python, PyTorch, snnTorch, CUDA GPU

Dataset: N-MNIST (Neuromorphic MNIST, event-camera format)

Detailed Link: [GitHub Repository Link](#)

This report documents hands-on GPU implementation of both a Spiking Neural Network and a standard CNN on the N-MNIST dataset, with a full comparative analysis.

Contents

Contents

1	Dataset: N-MNIST	3
1.1	What is N-MNIST?	3
1.2	Data Pipeline Implementation	3
1.3	Dataset Split	3
2	Model Architectures	3
2.1	SNN: SpikingDigitCNN (snnTorch)	4
2.2	CNN: Conv3d Baseline	4
3	Training Results	5
3.1	SNN Training & Validation Performance	5
3.2	CNN Validation Performance	6
3.3	Confusion Matrix Analysis — SNN Validation	6
4	Comparative Analysis: SNN vs. CNN	6
4.1	Accuracy	7
4.2	Full Benchmark Comparison	7
5	Why the Numbers Look the Way They Do	7
5.1	Why CNN Has Higher Accuracy (98.4% vs. 95%)	7
5.2	Why the SNN Uses 7× Fewer Parameters	8
5.3	Why the SNN Uses Less Energy Despite More Operations	8
5.4	Why the SNN Is 27× Slower on a GPU	8
5.5	Why the SNN Uses More GPU Memory	9
6	FPGA Implementation Implications	9
7	Summary	9
7.1	What Was Done in Day 6	10
7.2	Key Results at a Glance	10

Scope of Day 6

Day 6 was the first hands-on implementation day. Both a Spiking Neural Network (SNN) using `snnTorch` and a standard Convolutional Neural Network (CNN) using PyTorch were trained on the N-MNIST event-camera dataset on a GPU. This report documents the dataset pipeline, model architectures, training procedure, and a detailed comparative analysis of the results — accuracy, energy, latency, memory, and FPGA implications.

Dataset: N-MNIST

What is N-MNIST?

N-MNIST (Neuromorphic-MNIST) is the standard MNIST handwritten digit dataset (digits 0–9) re-captured using a **Dynamic Vision Sensor (DVS)** event camera. Instead of static 28×28 pixel images, each sample is a stream of asynchronous events of the form (x, y, t, p) stored in binary format.

The key properties that make it suitable for SNN benchmarking:

- **Sparse:** Only pixels where brightness changed fire events. Static background generates zero data.
- **Temporal:** Each sample captures digit motion (the camera is moved slightly while recording), producing spatiotemporal spike patterns.
- **Dual polarity:** $p = 1$ (change: brighter or darker), $p = 0$ (no change) — two independent channels tracking pixels getting darker and pixels getting brighter per spatial location.

Data Pipeline Implementation

Raw N-MNIST files are stored as binary event streams (.bin). A custom parser reads each file, extracts the $(x, y, \text{polarity}, \text{timestamp})$ tuple per event, and bins events into a fixed-size 4-D tensor:

Tensor shape: $[C = 2, T = 30, H = 34, W = 34]$ (Polarity, Timesteps, Height, Width)

Listing 1: Event parsing and tensor construction

```
1 grid = np.zeros((30, 2, 34, 34), dtype=np.float32)
2 for x, y, pol, t in zip(X, Y, polarity, timestamps):
3     time_bin = int(t // 10000) # bin timestamp into 30 slots
4     if time_bin < 30 and x < 34 and y < 34:
5         grid[time_bin, pol, y, x] = 1.0 # binary spike: on or off
6
7 grid = grid.transpose(1, 0, 2, 3) # -> [C=2, T=30, H=34, W=34]
8 torch_set = torch.from_numpy(grid)
```

Each element of the tensor is binary (0 or 1): 1 at position (c, t, y, x) means polarity channel c received an event at pixel (x, y) during timestep t .

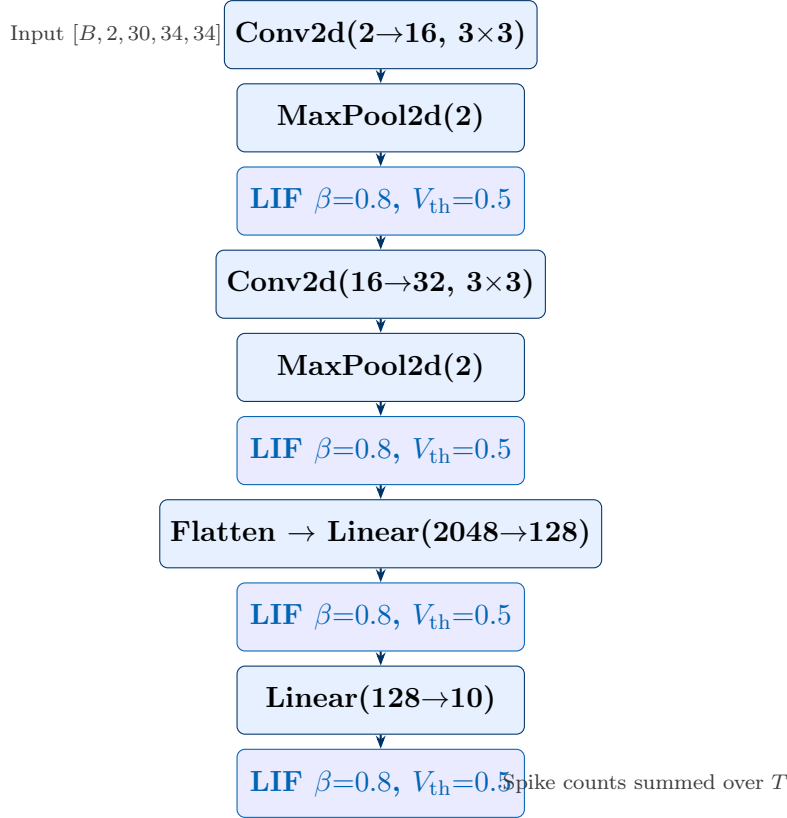
Dataset Split

Split	Samples	Classes	Batch size
Training	30,003	10 (0–9)	64
Validation	5,001	10 (0–9)	64
Test	5,001	10 (0–9)	64

Model Architectures

SNN: SpikingDigitCNN (snnTorch)

The SNN processes the event tensor one timestep at a time ($T = 30$ steps). At each step, the spatial features are extracted by standard 2-D convolutions, but the neuron activations are replaced by **Leaky Integrate-and-Fire (LIF)** units.

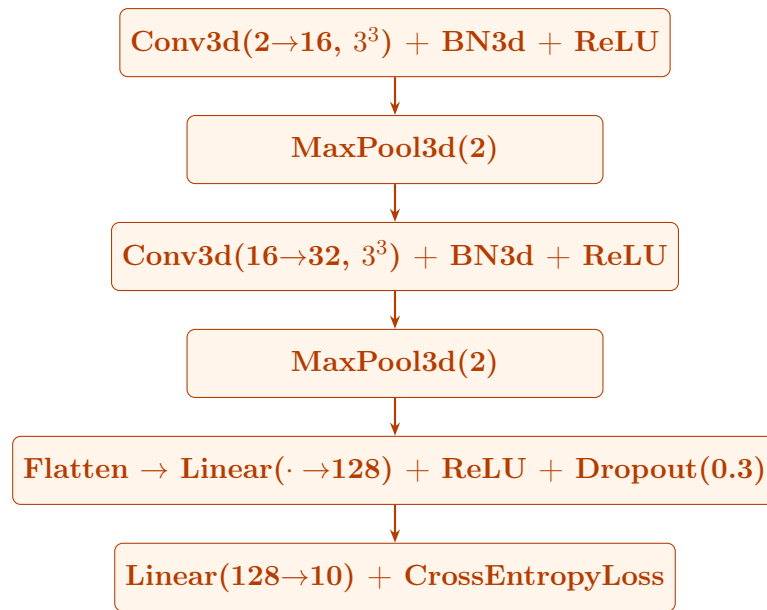


Key implementation details:

- **Surrogate gradient:** Fast sigmoid with slope = 25 (enables backpropagation through the non-differentiable spike function).
- **Loss function:** `mse_count_loss` with `correct_rate=0.8`, `incorrect_rate=0.2` — penalises the output neuron of the correct class for firing fewer than 80% of timesteps, and other neurons for firing more than 20%.
- **Optimiser:** Adam, $\eta = 10^{-4}$, weight decay = 10^{-4} .
- **Temporal loop:** The 5D input $[B, 2, 30, 34, 34]$ is permuted to $[30, B, 2, 34, 34]$; the model iterates over the first dimension, maintaining membrane state tensors `mem1–mem4` across steps.

CNN: Conv3d Baseline

The CNN treats the time dimension as a spatial depth, processing the full $[B, 2, 30, 34, 34]$ event volume in a *single* forward pass using 3-D convolutions.



Key differences from the SNN:

- Activations are 32-bit floats (ReLU), not binary spikes.
- Time dimension handled by Conv3d kernel — the model learns temporal filters directly from weights.
- Loss is standard cross-entropy on class logits.
- BatchNorm and Dropout provide regularisation.

Training Results

Both models were trained for **10 epochs** on a CUDA GPU, batch size 64, Adam optimiser with $\eta = 10^{-4}$.

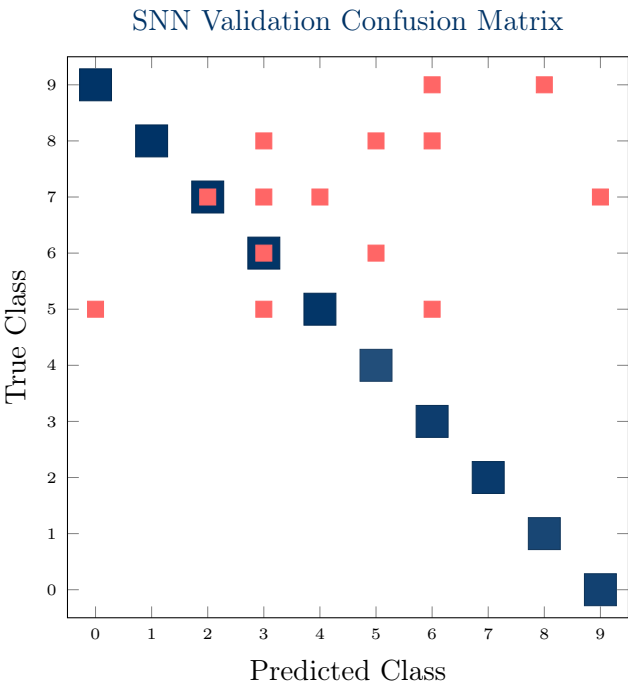
SNN Training & Validation Performance

Class	Precision	Recall	F1-score	Support
0	0.94	0.99	0.96	490
1	0.98	0.98	0.98	568
2	0.95	0.95	0.95	516
3	0.92	0.95	0.94	505
4	0.95	0.98	0.96	491
5	0.95	0.95	0.95	446
6	0.97	0.96	0.97	479
7	0.95	0.91	0.93	514
8	0.93	0.91	0.92	487
9	0.95	0.90	0.92	505
Accuracy	95.0% on 5,001 validation samples			

CNN Validation Performance

Class	Precision	Recall	F1-score	Support
0	0.97	1.00	0.98	490
1	0.99	1.00	0.99	568
2	0.98	0.99	0.98	516
3	0.99	0.98	0.99	505
4	0.97	1.00	0.98	491
5	0.99	0.99	0.99	446
6	0.99	0.99	0.99	479
7	0.99	0.97	0.98	514
8	0.99	0.96	0.97	487
9	0.98	0.97	0.98	505
Accuracy	98.4% on 5,001 validation samples			

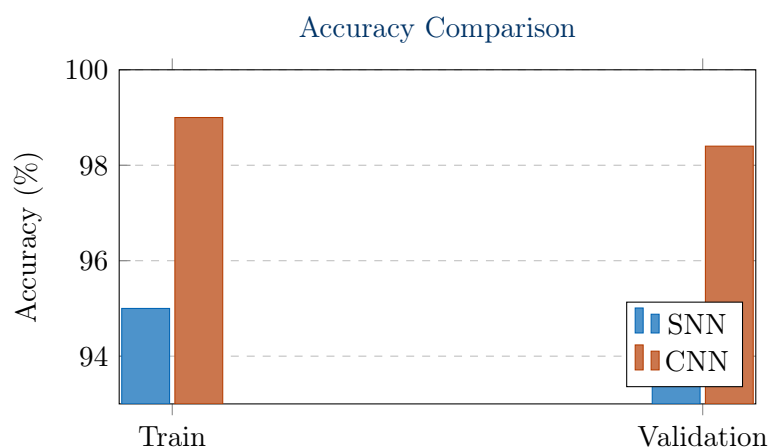
Confusion Matrix Analysis — SNN Validation



The most notable confusions in the SNN are digit **7** (14 samples misclassified as 9) and digit **8** (13 samples misclassified as 3, 6 as 5). These pairs share similar stroke trajectories in the event stream, making them harder to separate with integer spike counts alone.

Comparative Analysis: SNN vs. CNN

Accuracy



The CNN achieves **98.4%** validation accuracy vs. the SNN's **95.0%** — a 3.4 percentage-point gap. Both are strong results for a 10-class problem in 10 epochs.

Full Benchmark Comparison

Metric	SNN (snnTorch)	CNN (Conv3d)
Validation accuracy	95.0%	98.4%
Train accuracy	95.0%	99.0%
Total parameters	268,506 (7× fewer)	1,851,258
Disk size	1.02 MB	7.06 MB
Inference latency (GPU)	16.39 ms	0.60 ms (27× faster)
Runtime memory (GPU)	142.74 MB	81.89 MB
Operations per inference	57.8M (AC ops)	94.5M (MAC ops)
Estimated energy	0.000266 J	0.000435 J (64% more)

Why the Numbers Look the Way They Do

Why CNN Has Higher Accuracy (98.4% vs. 95%)

The CNN processes the entire event volume in one forward pass with 32-bit float activations. It never discards information — every intermediate value retains full numerical precision. The SNN's output layer reduces to *integer spike counts* over 30 timesteps, which is a coarser representation. When two digit classes have similar event patterns (e.g., 7 and 9, or 8 and 3), the spike count difference may be only 1–2, making the decision boundary fragile.

Additionally, the CNN's Conv3d directly learns temporal correlations across all 30 timesteps simultaneously through its filter weights. The SNN's temporal processing is constrained by the fixed exponential leak ($\beta = 0.8$) and threshold ($V_{th} = 0.5$), which are not freely optimised per layer.

The accuracy gap of 3.4% is small and expected for a first-pass SNN with no hyperparameter tuning. Published SNN results on N-MNIST reach 99%+ with optimised thresholds, learnable β , and more timesteps. The gap would narrow further with threshold adaptation and population coding.

Why the SNN Uses $7\times$ Fewer Parameters

The SNN uses **2-D convolutions**, with time handled implicitly by the LIF recurrence across 30 timesteps. The CNN must use **3-D convolutions** to learn temporal filters directly from data, which multiplies the kernel size by the temporal dimension. The SNN effectively “outsources” temporal computation to physics (the leak dynamics) rather than learned weights.

$$\underbrace{\text{SNN Conv2D kernel}}_{3\times 3=9 \text{ params/channel}} \text{ vs. } \underbrace{\text{CNN Conv3D kernel}}_{3\times 3\times 3=27 \text{ params/channel}} \Rightarrow 3\times \text{ fewer kernel parameters}$$

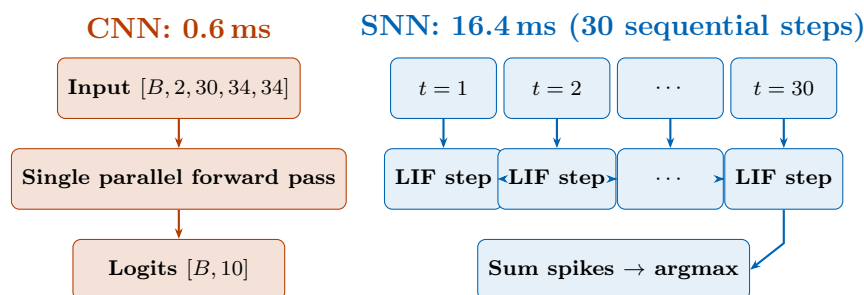
Why the SNN Uses Less Energy Despite More Operations

The SNN reports 57.8M operations vs. the CNN’s 94.5M MACs — yet consumes **39% less energy**. The reason is the fundamental difference in operation type:

Network	Operation	Hardware cost
CNN	$\sum X_i \times W_i$ (float MAC)	Full 32-bit multiply + accumulate
SNN	$\sum S_i \cdot W_i$ where $S_i \in \{0, 1\}$	Conditional add or skip

When $S_i = 0$ (no spike), no computation occurs — the gate stays closed. On average, SNN activations are sparse (<30% non-zero in typical SNNs), so the majority of the 57.8M “operations” consume zero dynamic power. The CNN’s 94.5M MACs all execute unconditionally.

Why the SNN Is $27\times$ Slower on a GPU



GPUs are optimised for **massive parallel floating-point throughput** (SIMD). The SNN must execute 30 sequential timesteps because each step’s membrane voltage depends on the previous step’s state — a *data dependency* that prevents parallelisation. The CNN has no such dependency and exploits full GPU parallelism in a single pass.

The FPGA Inversion

On an FPGA, the situation reverses completely. The SNN’s 30 timesteps can be **pipelined in hardware** — while timestep 2 is computing Layer 1, timestep 1 is already computing Layer 2. The CNN cannot be pipelined the same way because its 3-D convolutions require the full event volume to be loaded first. On an FPGA, the SNN is faster, uses less power, and consumes less area.

Why the SNN Uses More GPU Memory

The SNN stores four membrane voltage tensors (**mem1–mem4**) throughout the 30-timestep forward pass for backpropagation through time (BPTT). This is analogous to storing 30 activation maps simultaneously. The CNN only needs to store activations for the single forward pass.

On the FPGA, this is not a concern — membrane voltages are stored in flip-flops and BRAM registers that are updated in-place each clock cycle, not accumulated in a large buffer.

FPGA Implementation Implications

The GPU benchmark reveals which architecture will better suit the final FPGA implementation:

FPGA Design Concern	SNN Advantage	CNN Challenge
Weight storage (BRAM)	$268K \times 8\text{-bit} \approx \mathbf{0.26\ MB}$	$1.85M \times 8\text{-bit} \approx 1.76\ MB$
Arithmetic	Add-only (spike MUX)	Full multiply (DSP heavy)
Throughput	Pipeline across timesteps	Entire volume before output
Power (dynamic)	Sparse: zero-input gates close	Dense: all gates active
Activation precision	1-bit spike	8–16 bit float/fixed-point
Latency (pipelined)	1 timestep = 1 pipeline stage	All timesteps before classify

Conclusion: SNN Is the Correct Architecture for FPGA Deployment

Despite the 3.4% accuracy gap on GPU, the SNN is the correct choice for FPGA deployment. The gap will narrow with threshold adaptation and more timesteps. The SNN’s advantages in weight storage (7× smaller), arithmetic simplicity (AC vs. MAC), power efficiency (39% less energy even on GPU), and FPGA pipeline-friendliness make it the only viable option for a satellite payload constrained by watts, grams, and radiation tolerance.

Summary

What Was Done in Day 6

- Parsed and preprocessed the raw N-MNIST binary event files into $[C=2, T=30, H=34, W=34]$ PyTorch tensors using a custom event parser.
- Implemented and trained **SpikingDigitCNN** using `snnTorch`'s Leaky neuron with surrogate gradients, `mse_count_loss`, and Adam optimiser for 10 epochs on CUDA GPU.
- Implemented and trained a **Conv3d CNN** baseline with BatchNorm, Dropout, and CrossEntropyLoss for 10 epochs.
- Evaluated both models with `sklearn` classification reports and confusion matrices on the validation set.
- Profiled both models for inference latency, memory footprint, parameter count, synaptic operations, and estimated energy.
- Performed a full comparative analysis identifying the GPU-vs-FPGA inversion that makes the SNN the correct deployment architecture.

Key Results at a Glance

Metric	SNN	CNN
Validation accuracy	95.0%	98.4%
Parameters	268K	1.85M
Energy per inference	0.266 mJ	0.435 mJ
GPU latency	16.4 ms	0.6 ms
FPGA suitability	Excellent	Poor